COLLEGE OF ENGINEERING, GUINDY — ANNA UNIVERSITY, CHENNAI – 25

Course: Embedded Systems and Internet of Things | **Team: 14**

Railway Digital Twin with Wildlife Safety

An educational simulation integrating Godot 4, MQTT, a trusted Node.js control plane, React dashboards and ESP32 edge sensing

Team members:

Roll No.	Name	Roll No.	Name
2023103546	Kathir Kalidass B	2023103600	Parthasarathi S
2023103597	Malini R	2023103714	Paril T

Abstract

Abstract — A runnable digital twin of a wildlife-protected railway line: a Godot 4.7 3-D double-track "stadium" world with an island station, interlocked crossovers, level crossing, river bridges and Libre-TrainSim-derived asset fleet; a Node.js trusted control plane (JWT, RBAC, envelope validation, SQLite anti-replay and audit) that is the sole publisher of commands/state/signals on an authenticated Mosquitto MQTT broker with per-client ACLs; a React dashboard for viewer/operator/admin; an ESP32 edge node with fail-safe outputs verified in Wokwi over a Pinggy tunnel; and a FastAPI ML service (torchvision SSDLite) behind a two-frame camera-evidence policy. Wildlife protection escalates NONE→SUSPECTED (30 km/h)→CONFIRMED (0 km/h, STOP/RED) from trackside huts, an onboard simulated camera and ESP32 reports, with recovery-latched resets. Verification: Godot 42/42, backend 141/141, ML 8/8, ESP32 native 16/16, web build PASS, plus recorded live scenarios. Educational simulation only — not railway safety equipment.

1 I. Introduction and Objectives

Train–animal collisions on forested corridors motivate layered protection: *digital twins* let operators observe and rehearse operations while *embedded IoT edge nodes* sense hazards where they occur. Team 14 built a complete, runnable miniature railway covering both ideas. Objectives: (1) a 3-D double-track twin with real interlocking; (2) a trusted, audited, anti-replay control plane; (3) end-to-end security across REST and MQTT; (4) staged wildlife sensing (trackside hut, onboard camera, edge node) with two-frame camera confirmation; (5) an ESP32 fail-safe field node verified in a cloud simulator; (6) role-aware dashboards; (7) a continuous automated test gate. **Scope:** classroom simulation only; the Stage-D physical-hardware checklist is *not run*.

2 II. Architecture

One principle: *one trusted control plane, many restricted clients*. The Node backend is the sole MQTT publisher of `railway/state`, `events`, `alerts`, `signal/{block}` (retained), `commands/*` and `speed/*`. Mosquitto 2 (TCP 1883, WS 9001) authenticates four clients — `backend`, `godot`, `dashboard`, `esp32` — with generated credentials and per-client topic ACLs (Appendix matrix in the extended version; dashboard is read-only, ESP32 publishes only non-retained telemetry). Backend modules: `state.js`/`simulation.js` (two-train fleet: T1 UP clockwise, T3 DOWN anticlockwise; ten blocks), `signalling.js`/`routes.js` (per-block aspects, station-crossover interlock), `meets.js`/`rng.js` (seeded chance-meet scheduler), `wrong_line.js` (interlocked wrong-line run), `wildlife_commands.js`/`ingest.js`/`ml.js` (evidence fusion and camera ingest), `auth.js`/`rbac.js`/`replay.js`/`audit.js`/`db.js` (SQLite-backed security), `loop.js` (2 Hz fixed-timestep authority loop), `bus.js`/`mqtt.js` (retained publication). Failure posture: restrictive fail-safe on MQTT loss (`COMM_LOST`), retained snapshot reconcile on reconnect, recovery latch (`RECOVERY_REQUIRED`) until an authorised RESET, and no replay of stale envelopes or telemetry.

3 III. Workflow

Control path (REST): operator logs in (`/api/login`, bcrypt + JWT) → issues `/api/control`, `/api/route`, `/api/meet`, `/api/speed`, `/api/wrong-line` → backend validates role, envelope schema, ±5 min timestamp freshness and replay registry → publishes authoritative state (retained, 2 Hz). **Observation path (MQTT):** twin/ESP32 telemetry and wildlife evidence → backend fuses *trackside* + *onboard* evidence → block aspects (SUSPECTED→YELLOW, CONFIRMED→RED), alerts, ESP32 commands (`WILDLIFE_NEAR`, `WILDLIFE_DETECTED`, `RESET`) → dashboard WebSocket and twin reconciler update; ESP32 local inputs always dominate remote commands. Three independent protectors act on the same event: twin local limit (zero-latency braking even with MQTT offline), backend authority, and the edge node's own restrictive outputs. Demo: `./scripts/dev-up.sh` brings up the stack; `scripts/demo-scenarios.sh` drives the authenticated flow.

4 IV. Tools and Technology

Table 1: Technology stack

Layer	Technology (version / role)
Control plane	Node.js 22+, Express 4.21, better-sqlite3 13 (WAL), bcryptjs 3, jsonwebtoken 9, mqtt.js 5.16; Node built-in test runner (141 tests)
Live bus	Eclipse Mosquitto 2: authenticated MQTT, retained topics, per-client ACLs
Twin	Godot 4.7.2 (`gl_compatibility`; runs headless under `xvfb-run`); vendored `godot-mqtt` addon; 42-suite test harness
Assets	Blender-exported GLBs from Libre-TrainSim (train consist, station modules, deer/elephant) via `tools/export_lts_assets.py`; `test_real_assets` proves the exports replace procedural stubs
Dashboard	React 18.3.1 + Vite 6 + TypeScript 5.7 (`tsc -noEmit` gate)
Edge	PlatformIO espressif32 Arduino: PubSubClient 2.8, ArduinoJson 7, ESP32Servo, Adafruit SSD1306/GFX; Unity native tests
Edge sim	Wokwi (`diagram.json`, `wokwi.toml`, `scenarios/wildlife.yaml`, CLI v0.29.0); Pinggy raw-TCP tunnel to the local broker
ML	FastAPI 0.135.1 + uvicorn; torchvision 0.25 `ssdlite320_mobilenet_v3_large`, COCO animal classes; `ML_STUB=1` deterministic elephant 0.92 for integration runs
DevOps	Docker Compose, `dev-up.sh`, `verify-all.sh` gate

5 V. Peripherals (ESP32 Edge Node)

Field peripherals are wired in Wokwi and driven by the production firmware; the decision core lives in six pure-function headers (`sensors.h`, `debounce.h`, `output_logic.h`, `actuator.h`, `telemetry.h`, `command_logic.h`) so all logic is host-testable.

Table 2: Peripheral map (from `esp32/diagram.json`, 29 wires)

Peripheral	Pin	Function
Green / Red LED	GPIO26 / GPIO25	CLEAR vs RESTRICTIVE indicator
Buzzer	GPIO27	hazard alarm; `NEAR` pulsed warning
Wildlife button	GPIO33	field witness: animal at crossing
Fault button	GPIO32	sensor-fault witness; blocks RESET
HC-SR04 ultrasonic	GPIO5/18	obstacle distance (cm→m)
PIR motion	GPIO19	passive motion detection
Vibration (analog)	GPIO34	vibration threshold input
Barrier servo	GPIO13	boom 0° open / 90° restrictive
SSD1306 OLED	I^2C 21/22	status-line display

Sensor fusion: `animal_detected` = PIR ∨ distance≤threshold ∨ vibration≥threshold. Output precedence: `LOCAL WILDLIFE/FAULT` > `COMM LOSS` > backend stop > `ANIMAL NEAR` > `CLEAR` — local inputs are debounced and can never be cleared remotely. Telemetry carries `request_id`, zone, `device`, `sensorHealthy`, `wildlifeDetected`, `nearAnimal`, `uptimeMs`, `distanceM`, `motion`, `vibration`; SNTP-gated clock (`clockValid:false` + null timestamp until valid, which the backend rejects). Reconnect/publish is non-blocking; nothing is queued for replay.

6 VI. Hardware and Simulation

The ESP32 path is verified *without physical hardware* in two layers: (i) 16 Unity native cases (output precedence, debounce, telemetry serialization, command-zone validation) via `pio test -e native`, firmware builds `esp32dev` (Flash 87.1%); (ii) Wokwi cloud simulation of the exact `diagram.json` wiring running the same `firmware.bin`: `wokwi-tunnel.sh -write-secrets` opens a Pinggy raw-TCP tunnel and writes device-reachable endpoints into git-ignored `secrets.h`; `scenarios/wildlife.yaml` produces serial `MQTT connected` then `STATE LOCAL WILDLIFE`, observed live in the twin as `external=true` STOP/RED with the backend latched `RECOVERY_REQUIRED`. Stage-D checklist status: boot-restrictive and wildlife-input rows **PASS (Wokwi 2026-10-21)**; same/out-of-zone dashboard controls, reconnect behaviour and full serial inspection on physical hardware **not run**. Host evidence: Linux headless verification with Mesa llvmpipe (and WSLg D3D12 GPU at ≈11.8 ms/frame, 1280×720).

7 VII. Digital Twin World and Screenshots

The twin (`godot/scenes/lts_full.tscn`) renders a shared 250 m straight-sided stadium footprint (UP 1253.9822 m, DOWN 1228.8495 m; blocks 250.796/245.77 m), T1 clockwise and T3 anticlockwise with operational-chainage conversion (Fig. 1), an island platform spanning 150–198 m between the tracks inside `UP-B2`/`DOWN-B2` (Fig. 3), two interlocked station crossovers (X-A at 120 m, X-B at 240 m) used by meets and wrong-line runs (Fig. 4), a two-track wildlife level crossing with boom barriers and alarm, carved river with road + railway bridges, catenary, block signal masts, ten trackside sensor huts (75 m overlapping radius) and an onboard simulated camera. Keys: `SPACE` wildlife, `F` fault, `R` reset, `TAB` train switch, `C` camera, `N` day/night, `B` cinematic reel. All figures below were regenerated from `godot/tests/capture_*.gd` under headless software rendering (HUD shows `MQTT offline` in capture mode — expected).

8 VIII. Dashboards

Web dashboard (`web/src/App.tsx`): login with role-aware affordances (viewer observes; operator controls; admin inspects audit — enforced again server-side); top bar with MQTT pill and clock; hero actions (inject wildlife, simulate fault); fleet grid of per-train cards (current/target speed, occupied block, UP/DOWN pill, clockwise/anticlockwise); **Station Hold Scheduler** meet card (seed, phase, holder, route, countdown); **Wrong-Line Run** operator panel (eligibility, start, cancel; note "X-A entry / X-B exit" with opposing-train hold); **Sensor → Camera → Stop** wildlife panel (stage NONE/SUSPECTED/CONFIRMED, evidence sources, confidence, species, camera ONLINE/DEGRADED, ML STUB/LOADED, 30 km/h near warning); **block signalling & occupancy** diagram with per-block aspects and CAM/HUT evidence badges; **Audit & Operations Log** live event stream; **Wokwi ESP32** health card. **Twin-side HUD** mirrors the same backend state (zone, aspect, CAM state and confidence, sensor health,

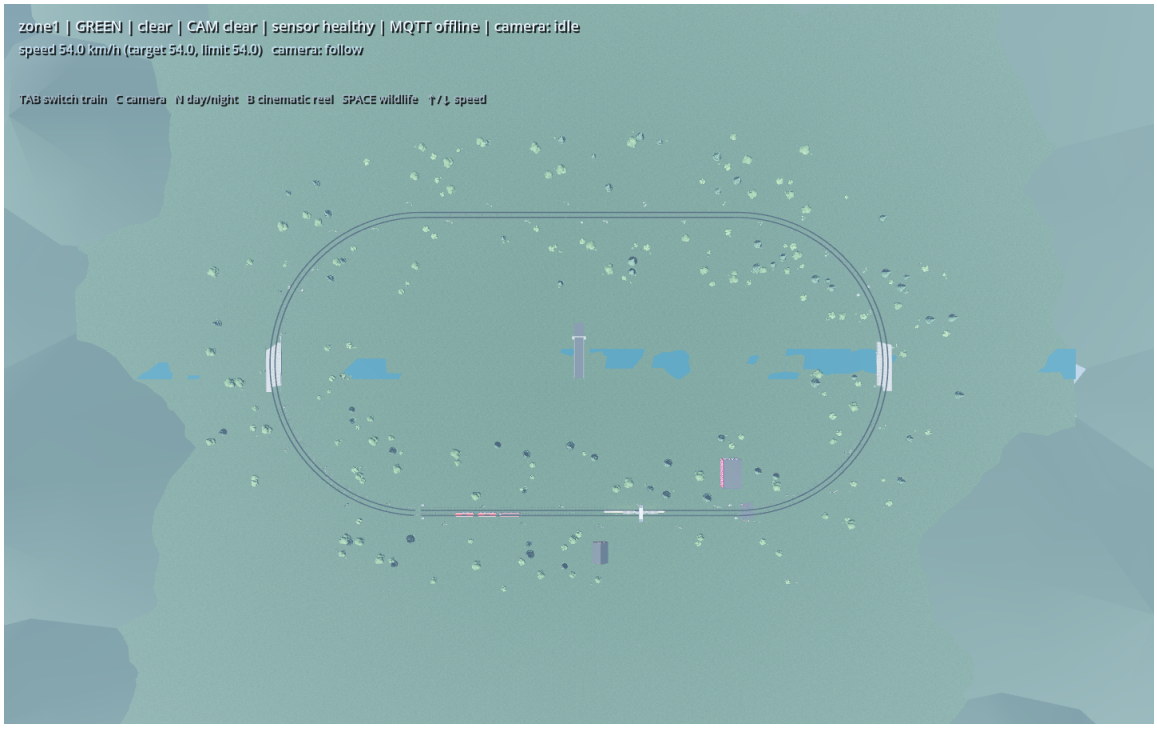


Figure 1: Top-down overview: stadium double track, island station, river with bridges, HUD state bar.



Figure 2: Confirmed wildlife stop: HUD `WILDLIFE STOP 0.95 / camera: CONFIRMED elephant 0.92`; target 0 km/h, elephant ahead, barrier down.

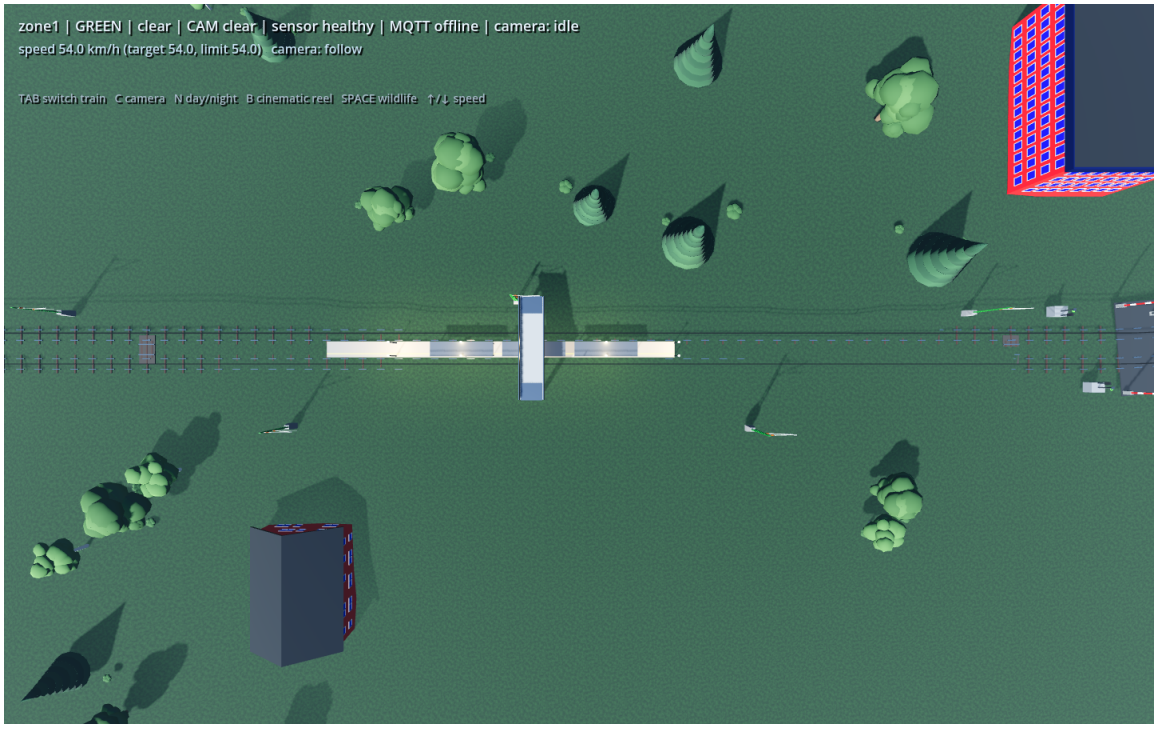


Figure 3: Island station between UP/DOWN tracks (top-down).

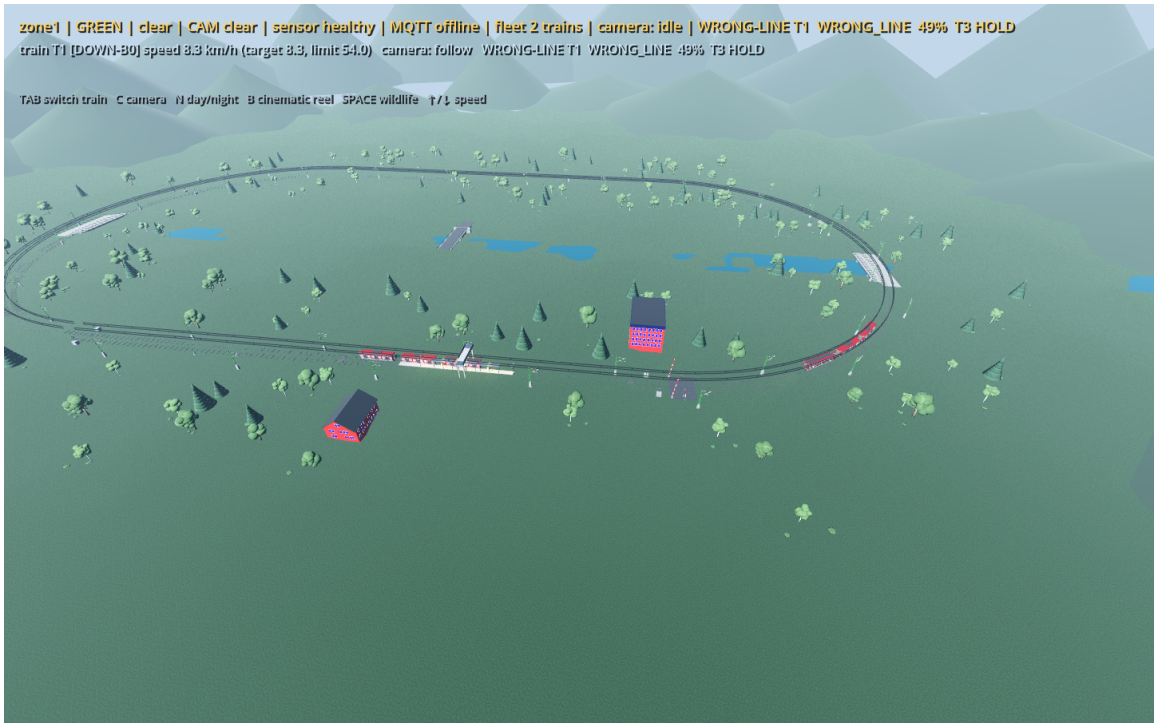


Figure 4: Interlocked wrong-line run: train blended onto the DOWN line at crossover A.

MQTT flag, speed/target/limit, meet and wrong-line HUD, camera-capture HUD in Fig. 2). Diagnostics: `GET /api/health` (`{"ok":true,"mqttConnected":true}`), `docker compose logs mqtt` for ACL denials, admin audit inspection.

9 IX. Features and Functions Implemented

Table 3: Feature inventory with verification evidence

Feature	Implementation / evidence
Auth + RBAC + audit	bcrypt/JWT login; viewer/operator/admin; audit row on every accepted/rejected/forbidden attempt (`auth/rbac/audit.js`; `auth.test.js`)
Envelope anti-replay	schema + ±5 min freshness + persistent `processed_requests` registry (`validate/replay.js`)
COMMS fail-safe	`COMM_LOST` state + retained reconcile + recovery latch requiring explicit RESET
Two-train fleet + 10 blocks	occupancy safety (never onto RED; own-overlap allowance), per-block aspects, retained signals (`signalling.test.js`, `simulation.test.js`)
Crossover interlock	one route at a time, hazard/occupancy 409s, audited `route:set` (`routes.test.js`)
Seeded meets	PRNG-reproducible hold + route execution (`meets/rng.test.js`)
Wrong-line run	phases APPROACH→CROSS_ENTRY→WRONG_LINE→CROSS_EXIT→CLEARANCE→DONE; opponent auto-held inside braking distance (live: T3 settled 320.1 m)
Distributed huts	10 huts, 75 m radius, status lamps, `railway/wildlife` observations (`test_sensor_huts`)
Onboard camera	forward cone ±25°/10°, 180–200 m, analytic occlusion (terrain/trunks/decks), confidence scoring (`test_onboard_detector`)
Stopping-distance model	$d = v\,t_r + v^2/2a + d_m$ (56 m at 36 km/h), safe-speed inversion (`test_onboard_braking`)
Staged hazard chain	trackside-only → SUSPECTED/YELLOW/30 km/h; two-frame camera confirm → CONFIRMED/RED/0 km/h; clear never downgrades; ESP32 `NEAR`/`DETECTED` delivery (`near_animal.test.js` 9/9, `camera.test.js` 9/9)
Camera ingest guards	bearer token, `X-Captured-At` freshness, `X-Block` targeting, duplicate-id suppression, 413 oversize, 429 rate (>5 fps), evidence cap 50 (`camera.test.js`)
ESP32 edge node	restrictive-first outputs, edge-triggered `WILDLIFE_NEAR`/`DETECTED`/`CLEARED`, zone-gated commands, buzzer pulses (16/16 native)
ML service	SSDLite MobileNetV3 COCO-animal detection; `ML_STUB` deterministic elephant; edge tests: 413, 503, invalid-image rejection (8/8 pytest)
Legacy hole removed	anonymous `POST /api/event` deleted (404); every control authenticated + audited

10 X. Results and Discussion

Final automated gate **(2026-10-23 closure audit)**: backend **141**/**141**, Godot **42**/**42**, ML **8**/**8**, ESP32 native **16**/**16**, firmware build **PASS** (Flash 87.1%), web build **PASS**. Recorded live runs: wrong-line request `WL-T1-1` held T3 at physical 320.1 m and released only on clearance; meet seed 4242 held T3 (48→0 km/h) with `HELD` outcome and identical history on identical seeds; near→confirm chain produced SUSPECTED (YELLOW, 30 km/h limit, `WILDLIFE_NEAR`) then CONFIRMED elephant 0.92 (RED, 0 km/h, `WILDLIFE_DETECTED`) with `RECOVERY_REQUIRED` latch and operator re-arm via `/api/speed`. Gate progression: backend 84→106→109→118→137→141; Godot 16→22→34→39→42; ESP32 native 14→16. **Limitations:** educational only; physical-hardware checklist not run; real ML weights not exercised (stub used); capture-mode frames are seconds apart so two-frame confirmation is verified over live HTTP instead. **Conclusion:** the architecture (one authority, many restricted clients, REST commands + retained MQTT state) survived every exercised failure mode and every subsystem is gated green; remaining gaps are exercise, not architecture.

11 References

[1] Godot Engine docs, 4.x, `https://docs.godotengine.org/`
[2] Eclipse Mosquitto 2 docs, `https://mosquitto.org/`
[3] OASIS, MQTT Version 5.0 Specification, 2019.
[4] HiveMQ, "MQTT Essentials" series (retained messages, QoS, topics), `https://www.hivemq.com/mqtt-essentials/`
[5] Wokwi docs (ESP32, `diagram.json`, CLI), `https://docs.wokwi.com/`
[6] PlatformIO docs (`platformio.ini`, Unity tests), `https://docs.platformio.org/`
[7] D. Hardt et al., "JSON Web Token Best Current Practices," RFC 8725, IETF, 2020.
[8] A. Howard et al., "Searching for MobileNetV3," ICCV 2019, pp. 1314–1324 (torchvision `ssdlite320_mobilenet_v3_large`).
[9] Express 4.x API docs, `https://expressjs.com/`
[10] React 18 docs, `https://react.dev/`
[11] Libre-TrainSim contributors, `https://github.com/JbDoe/libre-trainsim`
[12] SQLite WAL-mode docs, `https://www.sqlite.org/wal.html`
[13] FastAPI docs, `https://fastapi.tiangolo.com/`
[14] Pinggy docs (raw TCP tunnels), `https://pinggy.io/docs/`